Introduction to Machine Learning - II

```
$ echo "Data Sciences Institute - SUDS 2026"
$ https://github.com/upadhyan/SUDS-2026-Bootcamp-Friday
```

Goal

- Understand basic NLP concepts
- Learn use cases for embeddings
- Gain knowledge of the concept of foundation models and their applications

What is Natural Language Processing (NLP)?

How do we get computers to understand human language?

- Language is messy, ambiguous, and context-dependent.
- "I saw her duck" — did she duck, or did I see her pet duck?
- NLP is the branch of ML that deals with text and speech data.

NLP Tasks

Two broad categories:

- **Classification tasks**: assign a label to a piece of text.
- **Sequence-to-sequence tasks**: given text in, produce text out.

Classification Tasks

- **Text Classification**: Is this email spam or not spam?
- **Named Entity Recognition (NER)**: Identify people, places, organizations in text.
- **Part of Speech Tagging (POS)**: Label each word as noun, verb, adjective, etc.

Sequence to Sequence Tasks

- **Machine Translation**: English → French
- **Text Summarization**: Long article → short summary
- **Question Answering / Chatbots**: Question → Answer

The Representation Problem

All of our ML algorithms expect numbers as input.

Text is not numbers.

How do we convert text into something a model can work with?

Tokenization

Step 1: Break text into smaller units called **tokens**.

- A token can be a word, a subword, or a character.
- "I love machine learning" → `["I", "love", "machine", "learning"]`
- "unhappiness" → `["un", "happiness"]` (subword tokenization)

Tokenization

Why subwords?

- We can't store every word in every language in a vocabulary.
- Subword tokenization handles unseen words by breaking them into known pieces.
- Most modern models use subword tokenization (e.g. Byte-Pair Encoding).

Bag of Words (BoW)

Step 2: Represent a document as a vector of word counts.

1. Build a **vocabulary** of all unique words across your documents.
2. For each document, count how many times each word appears.

Bag of Words (BoW)

	"the"	"cat"	"sat"	"dog"	"ran"
Doc 1: "the cat sat"	1	1	1	0	0
Doc 2: "the dog ran"	1	0	0	1	1

- Simple, but ignores **word order** — "dog bites man" = "man bites dog".
- Common words like "the" dominate the representation.

TF-IDF

Term Frequency – Inverse Document Frequency

BoW, but with smarter weighting:

- **TF**: How often does this word appear in *this* document?
- **IDF**: How rare is this word across *all* documents?

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right)$$

- Common words (e.g. "the") get low weight; rare, meaningful words get high weight.

TF-IDF

	"the"	"cat"	"quantum"	"physics"
TF-IDF weights	Low	Medium	High	High

- Better than raw counts, but still ignores word order and meaning.
- "happy" and "joyful" are treated as completely unrelated.

Embeddings

```
$ echo "Questions?"
```

Embeddings

What if we could represent words as points in space, where similar words are close together?

- Map each token to a dense vector in a high-dimensional space (e.g. 300 dimensions).
- These vectors are learned from data, not hand-crafted.
- Examples: Word2Vec, GloVe, FastText.

Embeddings Contain Semantic Information

- Words with similar meanings are close in the embedding space.
- Embeddings capture relationships:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

- The model learns these relationships from context in large text corpora.

Document Embeddings

How do we go from word vectors to a vector for a whole document?

- **Simple approach**: Average all word embeddings in the document.
- **Better approaches**: Weighted average (using TF-IDF weights), or use a model that produces document-level embeddings directly.

Using Embeddings

Once text is a vector, you can apply any ML algorithm:

- **Classification**: Feed embeddings into a neural network or logistic regression.
- **Clustering**: Group similar documents using K-Means on their embeddings.
- **Search/Retrieval**: Find the most similar document using cosine similarity.

All the tools from Lecture 1 apply here!

Foundation Models

```
$ echo "Questions?"
```

Foundation Models

- **Large pre-trained models** trained on massive amounts of data.
- Can be adapted to many downstream tasks without training from scratch.
- Exist for text, images, audio, tabular data, and more.

Think of a foundation model as a university graduate with broad general knowledge. You can specialize them further for specific jobs.

Text Foundation Models

Core idea: given a sequence of words, predict the next word.

"The cat sat on the ___" → "mat"

- The model reads billions of sentences and learns patterns of language.
- This simple objective produces surprisingly powerful representations.

Text Foundation Models

Why do they work?

- Trained on enormous corpora (trillions of words from books, websites, code).
- Billions of parameters allow them to store complex patterns.
- Predicting the next word requires understanding grammar, facts, reasoning, and more.

We can extract embeddings from these models and use them for downstream tasks, or use the models directly.

Using Foundation Models Directly

Sometimes we don't need to extract embeddings. We can use the model as-is by providing instructions in natural language.

This is called **in-context learning**.

Zero-Shot Learning

Give the model a task description, no examples.

Prompt: "Classify the following review as positive or negative: 'This movie was absolutely terrible.'"

Output: "Negative"

- No training data needed.
- Works because the model has seen similar patterns during pre-training.

Few-Shot Learning

Give the model a task description **and a few examples**.

Prompt:
"Classify reviews as positive or negative.
'Great film!' → Positive
'Boring and slow.' → Negative
'I loved every minute.' → "

Output: "Positive"

- Providing examples helps the model understand the task format.

Zero-Shot vs. Few-Shot

Fine-Tuning

A foundation model is someone with a university education. They have broad general knowledge, but they are not a specialist. **Fine-tuning** is sending that person to graduate school.

Fine-Tuning

- $\theta = \{\theta_0, \theta_1, \ldots, \theta_n\}$ are the model's parameters.
- Index indicates the layer of the model.
- We can update only a subset, e.g. θ_n (last layer) or θ_{n-1}, θ_n (last two layers).

Fine-Tuning

- Avoids retraining the entire model from scratch (saves compute).
- Uses task-specific data to update the selected parameters.
- Use a **small learning rate** to avoid **catastrophic forgetting**: the model losing what it learned during pre-training.

Summary

Method	Data Needed	Compute	Flexibility
Zero-Shot	None	Low	Limited
Few-Shot	A few examples	Low	Moderate
Fine-Tuning	Task-specific dataset	High	High

Lab Time!

```
$ echo "Questions?"
```